

Plano de Arquitetura e Implementação — 1 Dia / 2 Pessoas

Assistente Pessoal Financeiro (APF) — Itaú Inovacamp | Serviço Único · SQLite · Dados Sintéticos Rotulados · Groq LLM

1. Visão Geral

Arquitetura mínima viável para ser construída em 1 dia por 2 pessoas. Eliminamos toda complexidade operacional desnecessária: um único serviço, um único banco de dados (SQLite), dados sintéticos explicitamente rotulados como tal e inferência via Groq, sujeita às cotas e condições vigentes do provedor.

- Serviço único: um backend em Python (FastAPI) que faz tudo — chat, decisão, EPC, LLM proxy.
- Banco único: SQLite (arquivo local) — zero configuração, zero Docker, zero container.
- Dados: sintéticos realistas, explicitamente rotulados como tal. Não reivindicamos acesso a APIs de produção do Itaú.
- LLM: Groq API — openai/gpt-oss-120b para linguagem, fallback 20b; qwen/qwen3.6-27b para visão de boleto quando disponível.
- Interfaces: Flutter com câmera, autenticação do dispositivo e deep links; Telegram com webhook autenticado e vínculo por código temporário.

Fora do escopo, deliberadamente: Redis, Kafka, Grafana, Docker Compose multi-container, PostgreSQL, TimescaleDB, Kubernetes.

2. Stack e Estrutura de Arquivos

Camada	Tecnologia	Por quê
Backend único	Python 3.11 + FastAPI	Um único main.py roda chat, EPC, decisão e LLM proxy. Sem microserviços.
Banco de dados	SQLite 3 (arquivo local)	Zero instalação, zero Docker. Backup é copiar o arquivo .db.
LLM	Groq API (gpt-oss-120b; fallback 20b)	Inferência hospedada; cotas, preço e latência dependem do plano vigente.
ORM	SQLAlchemy +aiosqlite	Async SQLite, models em Python.
App mobile	Flutter (1 tela)	Chat único via WebSocket para localhost:8000/ws.
Bot	Telegram Bot API + ngrok	Webhook HTTPS; túnel local apenas para demonstração.
Dados	Sintéticos (seed script)	Script Python gera 6 meses de transações realistas, rotuladas como sintéticas.

```
apf-hackathon/
├── main.py           # FastAPI: chat, ações, scanner e canais
├── transfers.py      # Pix/TED/boleto + estado transacional
├── document_intelligence.py  # Visão Groq + fallback de demo rotulado
├── telegram_integration.py  # Vínculo, webhook, handoff e tokens
├── models.py / database.py  # SQLite: dados, fluxos e identidades
├── decision.py / proactive.py  # Comparador, suitability e antecipação
├── llm.py / epc.py      # Linguagem com grounding + padrões SQL
├── configure_telegram.py  # Configuração do webhook
├── .env.example        # Variáveis sem segredos
└── flutter_app/        # Chat, câmera, biometria e app links
```

Para rodar: `pip install -r requirements.txt && python seed.py && uvicorn main:app --reload`. Em ~30 segundos, o serviço está no ar com dados sintéticos populados.

3. Serviço Único — main.py

O FastAPI expõe chat WebSocket e APIs para dashboard, padrões, privacidade, proatividade, scanner de boleto, vínculo Telegram, webhook e continuação segura.

```
@app.websocket('/ws')
@app.post('/user/{id}/boleto/scan')
@app.post('/user/{id}/proactive/scan')
@app.post('/user/{id}/telegram/link-code')
@app.post('/telegram/webhook')
@app.get('/continue/{token}')
```

```
# Regra central: canais externos iniciam a jornada;
# confirmação financeira acontece somente no app autenticado.
```

4. Dados Sintéticos — Honestidade Técnica

O protótipo não reivindica acesso a APIs de produção do Itaú. Usamos dados sintéticos realistas, explicitamente rotulados em toda resposta da API. Isso é honesto e evita perguntas sem boa resposta sobre como o time conseguiria acesso a APIs de produção de um banco em um hackathon.

```
# seed.py — 6 meses de transações sintéticas para 1 usuário demo
# Aluguel: PIX de R$ 1.500 para "João Silva", todo dia 10
# Salário: TED de R$ 5.500, todo dia 30
# iFood: ~R$ 45, 20 dias úteis/mês (delivery de almoço)
# Netflix: R$ 39,90, todo dia 15
```

```
// Toda resposta identifica a origem dos dados:
{
  "saldo": {"disponivel": 4523.87},
  "boletos": [{"id": "SABESP_001", "valor": 187.40}],
  "_source": "sintético"
}
```

```
// Scanner: "groq_vision" ou "demo_fallback" sempre visível ao usuário.
```

5. EPC — Detecção de Recorrência por Agregação SQL

Não usamos algoritmos de clustering (k-means, DBSCAN) nem bibliotecas de machine learning. A detecção de padrões é feita por agregação SQL simples: GROUP BY + HAVING + contagem de ciclos. Isso é suficiente para demonstrar o conceito de proatividade e é tecnicamente honesto.

```
-- Detecta Pix recorrente: mesmo favorecido, mesmo valor, 3+ meses
SELECT favorecido, valor, COUNT(*) as frequencia, MAX(data) as ultima_data
FROM transactions
WHERE user_id = ? AND tipo = 'pix' AND favorecido IS NOT NULL
GROUP BY favorecido, valor
HAVING COUNT(*) >= 3 AND (MAX(valor) - MIN(valor)) / MAX(valor) < 0.05

-- Detecta gasto fixo recorrente: mesma categoria, mesma faixa de valor, 12+ ocorrências
SELECT categoria, AVG(valor) as valor_medio, COUNT(*) as frequencia
FROM transactions
WHERE user_id = ? AND data > date('now', '-3 months')
GROUP BY categoria, CAST(strftime('%w', data) AS INTEGER)
HAVING COUNT(*) >= 12 AND (MAX(valor) - MIN(valor)) / MAX(valor) < 0.20
```

Método rotulado explicitamente na resposta: `"_metodo": "agregacao_sql"`, `"_nota": "Não utiliza clustering estatístico."`

6. Suitability Gate (Resolução CVM 30)

Qualquer recomendação de produto de investimento no Brasil está sujeita à Resolução CVM 30, que exige análise de perfil de investidor. O protótipo implementa um gate explícito:

- Perfil armazenado no SQLite: conservador, moderado ou arrojado.
- Produtos permitidos no protótipo (dispensam suitability complexo): CDB liquidez diária, poupança, Tesouro Selic.
- Produtos bloqueados no protótipo: ações, fundos multimercado, cripto, COE — qualquer coisa além de renda fixa simples.
- Disclaimer obrigatório em toda projeção: “Projeção estimada. Rentabilidade passada não garante rentabilidade futura.”

```
PERFIL_PRODUTOS = {
    'conservador': ['cdb_liquidez', 'poupanca', 'tesouro_selic'],
    'moderado': ['cdb_liquidez', 'poupanca', 'tesouro_selic', 'cdb_pos'],
    'arrojado': ['cdb_liquidez', 'poupanca', 'tesouro_selic', 'cdb_pos', 'fundos_rf'],
}

async def check_suitability_gate(user_id, produto):
    perfil = await get_perfil(user_id)
    if produto not in PERFIL_PRODUTOS.get(perfil, []):
        return {'permitido': False,
                'mensagem': 'Produto não disponível para seu perfil. Sugiro falar com um assessor Itaú.'}
    return {'permitido': True}
```

7. Consentimento LGPD — Granular por Categoria

Construir um perfil comportamental (onde almoça, quando paga aluguel, quanto sobra de salário) é processamento de dado sensível por inferência. O protótipo exige consentimento granular por categoria, não um checkbox genérico de “aceito os termos”.

- 3 checkboxes independentes no onboarding, desmarcados por padrão: (1) analisar padrões de pagamento recorrente; (2) analisar hábitos de gasto; (3) monitorar saldo ocioso para sugerir investimento.
- Opt-out por categoria a qualquer momento, no menu “Privacidade”.
- Direito ao esquecimento: botão “Excluir meus dados” apaga transações, padrões e conversas do SQLite.
- Transparência: toda notificação inclui “Baseado no seu padrão de [categoria]”.

8. Fadiga de Notificação — Cap Diário e Agrupamento

Um agente que envia mensagens de alerta, insight e sugestão todo santo dia é exatamente o tipo de coisa que vira “mais um app que eu silencie”. Regras explícitas de frequência:

- Cap diário: máximo de 2 notificações proativas por dia por usuário.
- Janela de envio: 9h às 20h. Fora disso, fica na fila até o dia seguinte.
- Agrupamento inteligente: múltiplos padrões no mesmo dia viram uma única mensagem consolidada.
- Cooldown por categoria: 7 dias após Pix recorrente, 14 após insight de economia, 30 após investimento.
- Silenciamento fácil: botão “Não me avise mais sobre isso” em toda notificação.
- Métrica de saúde: se a taxa de dismiss passar de 50% numa categoria, a frequência dessa categoria cai automaticamente pela metade.

```
MAX_NOTIFICACOES_DIA = 2
JANELA = (9, 20)
COOLDOWNS = {'pix_recorrente': 7, 'economia': 14, 'investimento': 30}
```

```

async def send_proactive_notification(user_id, categoria, mensagem):
    if not await can_notify(user_id, categoria): return {'enviado': False, 'motivo': 'sem_consentimento'}
    if user.notificacoes_hoje >= MAX_NOTIFICACOES_DIA: return {'enviado': False, 'motivo': 'cap_diario'}
    if not dentro_da_janela(): return {'enviado': False, 'motivo': 'fora_da_janela'}
    if em_cooldown(user_id, categoria): return {'enviado': False, 'motivo': 'cooldown'}
    await push_notification(user_id, mensagem)
    return {'enviado': True}

```

9. Flutter — Chat em 1 Tela

ChatScreen usa WebSocket, cards acionáveis e scanner por câmera/galeria. local_auth solicita autenticação real em dispositivo compatível; web e ambientes sem suporte exibem fallback de demonstração rotulado. app_links retoma tokens do Telegram. A proatividade é verificada automaticamente após a conexão. Onboarding mantém três consentimentos LGPD granulares.

10. Telegram — Estrutura Pronta para Integrar

- Preencher TELEGRAM_BOT_TOKEN, TELEGRAM_BOT_USERNAME, TELEGRAM_WEBHOOK_SECRET e PUBLIC_BASE_URL no .env.
- Executar python configure_telegram.py para registrar o webhook HTTPS.
- Gerar POST /user/{id}/telegram/link-code; o cliente abre o link t.me e vincula o chat com código de uso único.
- Ações financeiras geram botão “Continuar com segurança no app”, token de 15 minutos e deep link. O Telegram nunca executa dinheiro.

11. Checklist — Do Zero ao Demo em 1 Dia

Horário	Pessoa	Tarefa
0–2h	Dev 1 (Backend)	Setup SQLite, models, seed.py com dados sintéticos. main.py com endpoints /ws e /telegram/webhook.
0–2h	Dev 2 (Flutter)	Criar projeto Flutter, tela de chat, WebSocket conectando em localhost:8000/ws.
2–4h	Dev 1	Integrar Groq (llm.py). Prompt engineering + guardrails + function calling básico.
2–4h	Dev 2	Rich cards no Flutter: comparador de pagamento, biometria simulada, onboarding LGPD.
4–6h	Dev 1	EPC (epc.py): agregação SQL detectando recorrências. Notificações com regras de frequência.
4–6h	Dev 2	Telegram bot com ngrok. Deep link para ações financeiras. Teste end-to-end.
6–8h	Ambos	Polimento: suitability gate, pílulas de conhecimento JSON, tratamento de erro, animações.
8h+	Ambos	Gravação do pitch, testes finais, documentação.

12. Conclusão — Honestidade Técnica como Diferencial

A arquitetura continua deliberadamente simples, mas agora cobre a jornada completa: estado persistente, scanner multimodal, autenticação do dispositivo e handoff Telegram → app. Componentes de demonstração são substituíveis por serviços Itaú sem alterar contratos ou regras de negócio.

- Dados: sintéticos, realistas, explicitamente rotulados. Nenhuma reivindicação falsa de acesso às APIs do Itaú.
- EPC: detecção de recorrência por agregação SQL, não clustering estatístico. Terminologia honesta.
- Suitability: gate explícito para CVM, apenas produtos de baixíssimo risco, disclaimer obrigatório.
- LGPD: consentimento granular por categoria, opt-out fácil, direito ao esquecimento implementado.
- Fadiga: cap diário, janela, cooldown por categoria e silenciamento em 1 clique estão implementados; agrupamento e adaptação por dismiss são evoluções futuras.
- Custo de infraestrutura local próximo de zero, sujeito às cotas e condições dos provedores externos. Escalabilidade futura: trocar SQLite por banco gerenciado e adicionar filas/observabilidade sem reescrever a lógica de jornada.

13. Critérios de Aceite da Integração

Antes da demonstração

Backend saudável e base sintética carregada.
Build Flutter validada; câmera e autenticação conferidas no dispositivo.
Bot vinculado ao usuário demo e webhook HTTPS configurado.
Boleto sintético “não pagável” disponível na galeria.

Antes de produção

Desativar TELEGRAM_ALLOW_DEMO_USER e exigir vínculo verificado.
Usar cofre de segredos, atestação de dispositivo, idempotência distribuída e auditoria.
Substituir dados sintéticos e execução local por APIs e sandbox Itaú.